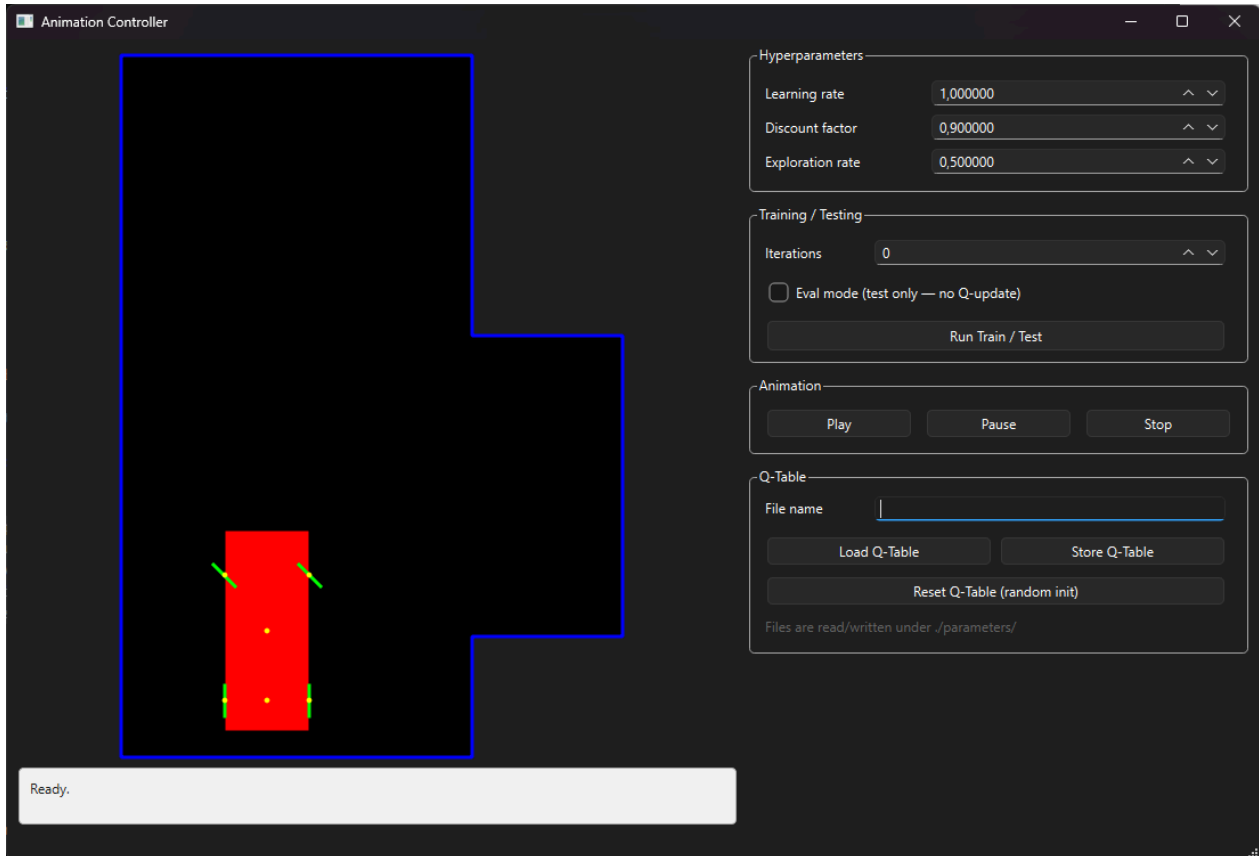


Car Parking with Q-learning

Daniele Pisani

daniele.pisani@santannapisa.it

2026-05-10



Abstract

Autonomous parking is a complex task that requires an agent to learn how to control a car in a constrained environment. In this project, we implemented a simple 2D simulation of a car and trained a Q-learning agent to park it in a designated spot. The car's state is defined by its position and orientation while the action space consists of discrete steering angles and acceleration commands. We experimented with different reward functions and hyperparameters to optimize the learning process. The results show that the agent successfully learns to park the car in the target spot after sufficient training episodes, demonstrating the effectiveness of Q-learning in this context.

Index

1 Introduction	3
2 Physical Modeling	3
2.1 State and inputs	3
2.2 Exact bicycle kinematics	3
2.3 Linearized model	4
2.4 Body geometry and collisions	4
3 Learning architecture	5
3.1 State and action space	5
3.2 Reward and termination	6
3.3 Q-learning update	6
4 Implementation	7
4.1 Tools	7
4.2 Organization	7
4.3 Hyperparameter	8
5 Experiments and results	8
5.1 Various hyperparameters and training dimensions tested (maybe removable)	9
6 Conclusions	10
7 Further Work	10

1 Introduction

The aim of this project is training a small Q-learning model to park a car in a designated spot. Here, the car is a simplified 2D model, designed as a rectangle with a front pair of rotating wheels, and a back pair of fixed wheels. The states of the car are defined by its position and orientation. The action space consists of discrete steering angles and speed commands. The environment is a simple parking lot with a designated parking spot. The agent receives rewards based on its ability to successfully park within it while avoiding collisions in as little time as possible.

The choice of method is deliberately conservative. Tabular Q-learning is the simplest possible reinforcement-learning algorithm with provable convergence on finite Markov decision processes, and it serves as a transparent baseline against which any more sophisticated approach can be compared. The cost of this simplicity is paid in the discretization step: the continuous pose of the car must be projected onto a finite grid, and every design decision in Section 2 and Section 3 is shaped by the trade-off between resolution and table size. The remainder of this paper is organized as follows. Section 2 describes the kinematic-bicycle simulator and the geometry of the parking lot. Section 3 defines the discrete state and action spaces, the reward function, and the Q-learning update used by the agent. Section 4 discusses how the simulator and the learner are integrated into a single Qt application, which hyperparameters are exposed, and how the Q-table is persisted. The final two sections summarize qualitative observations and possible extensions.

2 Physical Modeling

The car's dynamics are modeled using a simplified kinematic bicycle model, which captures the essential behavior of a car while being computationally efficient. The speed is modeled instantaneously, meaning that the car can accelerate or decelerate to any speed within a single time step. The steering is also modeled as instantaneous, allowing for discrete changes in the steering angle. The fact that the wheels, specifically the front ones, are not on the edge of the car, but rather a few centimeters inwards, must be taken into account in the model to allow for the parking to be possible.

2.1 State and inputs

The car's pose is described by the triple (x, y, θ) , where (x, y) is the position of the geometric centre of the rectangular body and θ is the heading angle in world coordinates. At each step the controller issues two signals: a longitudinal speed v and a front-wheel steering angle δ , both drawn from a discrete set symmetric around 0.

Let L_c be the total length of the car, w_c its width, and o_f, o_r the front and rear overhangs. The effective wheelbase, i.e. the distance between the front and rear axles, is then

$$L = L_c - o_f - o_r.$$

For the exact kinematic model, the rear-axle contact point (x_r, y_r) is located at a distance $\frac{L_c}{2} - o_r$ behind the body centre along the heading direction:

$$\begin{pmatrix} x_r \\ y_r \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} - \left(\frac{L_c}{2} - o_r \right) \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}.$$

2.2 Exact bicycle kinematics

With a non-zero steering angle, the car traces a circular arc around the instantaneous centre of rotation (ICR), which lies on the line perpendicular to the heading at the rear axle, at a signed distance $\frac{L}{\tan \delta}$. Over a time step Δt , the swept arc angle is

$$\alpha = \frac{\Delta t v \tan \delta}{L},$$

and the ICR is

$$\begin{pmatrix} x_{cr} \\ y_{cr} \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} - \left(\frac{L_c}{2} - o_r \right) \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix} - \frac{L}{\tan \delta} \begin{pmatrix} \sin \theta \\ -\cos \theta \end{pmatrix}.$$

The new pose is obtained by rigidly rotating (x, y, θ) around (x_{cr}, y_{cr}) by α :

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \begin{pmatrix} x - x_{cr} \\ y - y_{cr} \end{pmatrix} + \begin{pmatrix} x_{cr} \\ y_{cr} \end{pmatrix}, \quad \theta' = \theta + \alpha.$$

When $\delta = 0$ the ICR escapes to infinity, so the update reduces to pure translation along the heading:

$$x' = x + \Delta t v \cos \theta, \quad y' = y + \Delta t v \sin \theta, \quad \theta' = \theta.$$

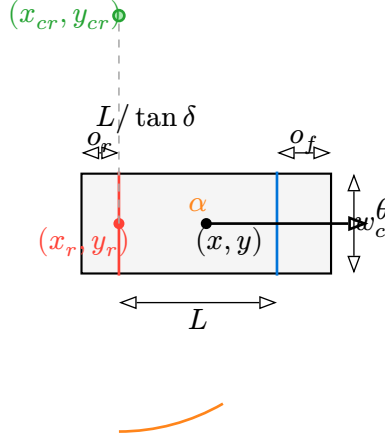


Figure 1: Kinematic bicycle model. The body of length L_c and width w_c has wheelbase $L = L_c - o_f - o_r$, with overhangs o_f and o_r at the front and rear. The rear axle (x_r, y_r) lies at distance $\frac{L_c}{2} - o_r$ behind the geometric centre (x, y) . For non-zero steering δ , the body rotates around the instantaneous centre of rotation (x_{cr}, y_{cr}) , perpendicular to the heading at signed distance $L / \tan \delta$ from the rear axle; the swept arc over one timestep is α .

2.3 Linearized model

A simpler integrator is also implemented (selected by the APPROX_MOTION flag in the configuration). It tracks a rear reference point $(\tilde{x}_r, \tilde{y}_r)$ taken at the midpoint of the rear bumper rather than at the rear axle (the overhang o_r is ignored here, so this point is offset by $\frac{L_c}{2}$ instead of $\frac{L_c}{2} - o_r$):

$$\begin{pmatrix} \tilde{x}_r \\ \tilde{y}_r \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} - \frac{L_c}{2} \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}.$$

This point is translated straight along the current heading and the heading is updated independently, dropping the curvature coupling:

$$\begin{pmatrix} \tilde{x}_r' \\ \tilde{y}_r' \end{pmatrix} = \begin{pmatrix} \tilde{x}_r \\ \tilde{y}_r \end{pmatrix} + \Delta t v \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix}, \quad \theta' = \theta - \frac{\Delta t v \sin \delta}{L_c}.$$

This trades geometric fidelity for speed: it replaces $\tan \delta$ with $\sin \delta$ and uses the full body length L_c in place of the wheelbase L , so the predicted curvature is biased for large steering angles. The exact model is the default.

2.4 Body geometry and collisions

The four corners (x_i, y_i) , $i = 1, \dots, 4$, of the car rectangle are reconstructed independently from the pose, each as the body centre plus a half-length offset along the heading and a half-width offset perpendicular to it:

$$\begin{pmatrix} x_i \\ y_i \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} \pm \frac{L_c}{2} \begin{pmatrix} \cos \theta \\ \sin \theta \end{pmatrix} \pm \frac{w_c}{2} \begin{pmatrix} \sin \theta \\ -\cos \theta \end{pmatrix},$$

with the four sign combinations giving the four vertices. A configuration is admissible when all four corners lie inside the parking-lot polygon and the body does not swallow either of the two re-entrant corners of the slot pocket. The car is considered parked when all four corners lie inside the slot quadrilateral and the heading satisfies

$$|\theta - (-\frac{\pi}{2})| \leq \frac{\pi}{6}.$$

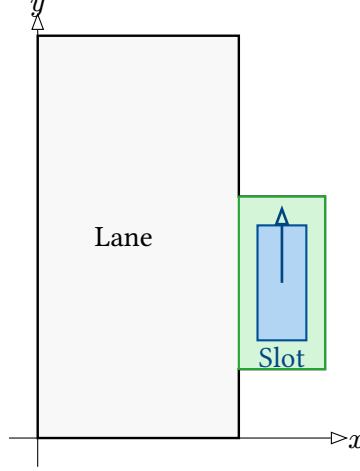


Figure 2: Parking lot polygon (light grey) and target slot quadrilateral (green), drawn from configuration/default.conf with $W_{\text{env}} = 10$, $L_{\text{env}} = 14$, $W_{\text{slot}} = 3$, $L_{\text{slot}} = 6$, and $\text{FREE_PARK} = 0.4$. The blue rectangle shows the canonical parked configuration, with heading $\theta = -\frac{\pi}{2}$ in the simulator’s y -down convention (mirrored here for readability so that y points up).

3 Learning architecture

This chapter describes how the continuous parking task introduced in Section 2 is reformulated as a finite Markov decision process suitable for tabular Q-learning. The three ingredients are a discrete state space obtained by binning the pose of the car, a discrete action space obtained by sampling speed and steering on a regular grid, and a sparse reward function that distinguishes only three outcomes per step.

3.1 State and action space

The state of the agent is the discretized pose of the car. The continuous triple (x, y, θ) is binned independently into X_{DIVIDE} position bins along the lot width, Y_{DIVIDE} position bins along its length, and Θ_{DIVIDE} angular bins covering $[0, 2\pi)$, yielding

$$N_{\text{states}} = X_{\text{DIVIDE}} \cdot Y_{\text{DIVIDE}} \cdot \Theta_{\text{DIVIDE}}$$

discrete states. The state index packs the three components with θ as the fastest-varying axis, so that nearby orientations occupy contiguous indices in the table:

$$s = i_{\theta} + \Theta_{\text{DIVIDE}}(i_y + Y_{\text{DIVIDE}} i_x).$$

The bin indices i_x, i_y, i_{θ} are obtained by integer division of each coordinate by the corresponding section width, with θ wrapped into $[0, 2\pi)$ before binning. Cells whose index would overflow are clamped to the largest valid bin so that no transition can ever produce an out-of-range state.

The action space is the Cartesian product of a discrete speed set and a discrete steering set, both built by a small helper that produces n values symmetric around zero with a fixed spacing. Speeds are spaced in

metres per second, steering values are spaced by a configurable angular unit and converted from degrees to radians before use. The total number of actions is

$$N_{\text{actions}} = N_{\text{speed}} \cdot N_{\text{steering}},$$

and a flat action index a decomposes as $a_{\text{speed}} = a \div N_{\text{steering}}$, $a_{\text{steering}} = a \bmod N_{\text{steering}}$. The presence of a zero entry in both sets is essential: it lets the agent stand still or drive straight, which is required for the fine-grained alignment manoeuvres performed near the slot.

3.2 Reward and termination

The reward function is deliberately sparse and depends only on the geometric classification of the resulting pose. After applying a transition, the environment classifies s' into one of three regimes and emits a constant reward:

$$r(s, a, s') = \begin{cases} R_{\text{hit}} & \text{if any car corner exits the lot polygon} \\ R_{\text{park}} & \text{if the car is parked} \\ R_{\text{nothing}} & \text{otherwise.} \end{cases}$$

With $R_{\text{hit}} < 0$, $R_{\text{park}} > 0$ and R_{nothing} either zero or a small negative time penalty, this scheme penalizes collisions, rewards success, and lets the discount factor γ implicitly favour shorter trajectories. The “parked” predicate combines two conditions, a geometric one and an angular one: the four corners of the body must lie inside the slot quadrilateral and the heading must satisfy $|\theta - (-\frac{\pi}{2})| \leq \frac{\pi}{6}$, as derived in Section 2. No explicit episode termination is enforced. The simulator simply continues, and the headless training loop relies on the reward signal alone to drive the policy away from collisions and toward the slot.

3.3 Q-learning update

The agent maintains a tabular action-value function $Q \in \mathbb{R}^{N_{\text{states}} \times N_{\text{actions}}}$, initialized to zero. At every model step the standard tabular Q-learning update is applied:

$$Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right),$$

with learning rate η and discount factor γ . Action selection uses an ε -greedy policy:

$$\pi(s) = \begin{cases} \text{uniform random action with probability } \varepsilon \\ \arg \max_a Q(s, a) & \text{with probability } 1 - \varepsilon. \end{cases}$$

During evaluation the random branch is suppressed by setting $\varepsilon = 0$. This is exposed in the user interface through an `eval` checkbox: ticking it bypasses the Q-update inside the iteration step and forces purely greedy action selection, turning the agent into an exploiter of the current table without overwriting it.

Both η and ε decay over training. Each parameter is configured by a maximum value, a minimum value, and a half-life H expressed in model iterations. Every 1000 iterations the parameter is multiplied by a constant ratio $\rho = 2^{-\frac{1000}{H}}$ derived from the half-life, so that after H iterations the gap between the current value and the asymptotic floor has halved:

$$\eta_t - \eta_{\min} \approx (\eta_{\max} - \eta_{\min}) \rho_{\eta}^{\frac{t}{1000}}, \quad \rho_{\eta} = 2^{-\frac{1000}{H_{\eta}}},$$

and analogously for ε . This schedule lets the agent explore aggressively early in training while gradually committing to the policy implied by the current Q-values, and prevents stale large-step updates from overwriting a good table once the policy has approximately converged.

4 Implementation

The full project is implemented as a single Qt 5/6 desktop application written in C++. The same binary handles training and visualization, and switching between the two modes is done through the user interface rather than through separate executables. There is no external machine-learning framework: the Q-table, the simulator, and the rendering pipeline are all hand-written in a few thousand lines of code.

4.1 Tools

Building requires Qt (modules core, gui and widgets) together with qmake and a make-compatible tool-chain. From the project root the standard sequence is `qmake carparking.pro` to generate the Makefile, then `make` (or `mingw32-make` on Windows, `nmake` under MSVC) to compile, and finally `./bin/carparking` to run. Object files are emitted to `build/` and the executable to `bin/`, controlled by `OBJECTS_DIR` and `DESTDIR` in the project file. No automated test harness, linter, or continuous-integration pipeline is configured: changes are validated by building and running the GUI.

The application also relies on three paths resolved relative to the working directory: `configuration/default.conf` is read once at startup, `parameters/` stores serialized Q-tables, and `log/log12.txt` receives the periodic training log written by the headless training loop. Launching the binary from a different directory silently breaks these reads and writes, so the project root must always be the current directory. Documentation can be regenerated with `doxygen Doxyfile` and is emitted to `docs/html/`.

4.2 Organization

The codebase is organized around three components that mirror the natural separation between simulator, learner, and presentation:

- `Environment` and `CarState` (`src/environment.{h,cpp}`) implement the kinematic model described in Section 2, the parking-lot polygon, the collision and parking checks, the reward function, and the state discretization. The class name uses the project’s canonical (mis)spelling and is preserved consistently throughout the source.
- `QLearningModel` (`src/q_learning.{h,cpp}`) owns the flat Q-table as a `vector<vector<float>>` of shape $[N_{\text{states}}, N_{\text{actions}}]$, the ϵ -greedy action selection, the tabular update, and the schedules for η and ϵ . A single instance handles the full action space; although the user interface labels separate “speed” and “steering” controllers, in practice both are encoded in the same flat action index.
- `MainWindow` (`src/mainwindow.{h,cpp}` and `mainwindow.ui`) owns one `Environment` and one `QLearningModel`, drives both execution paths, performs all painting (no separate view widget exists), and exposes the buttons that load and store the Q-table.

The application supports two distinct execution paths, each driven by its own button. *Animated mode* advances the simulation with a `QTimer`: every tick steps the environment by one timestep, and the model is updated only every `time_ratio` ticks, which lets the visual pace differ from the learning pace. This path is meant for visual debugging and qualitative inspection of a trained policy, not for serious training. *Headless training*, by contrast, enters a tight for loop over model iterations with no timer, alternating one Q-learning update with one environment step on every iteration. This is the only path that prints aggregated statistics to the standard output and writes the structured log to `log/log12.txt` every `LOG_FREQ` iterations, and is the path used to actually train a policy from a cold start.

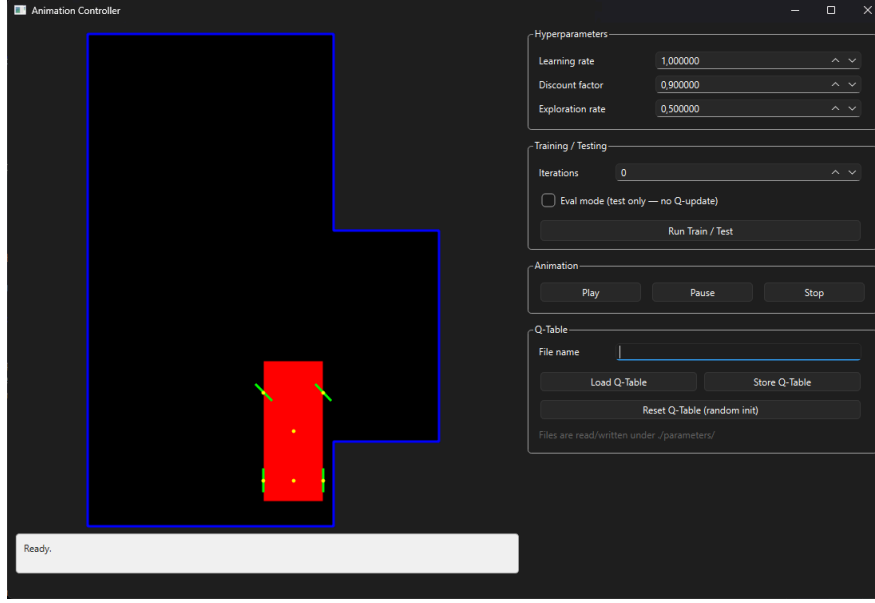


Figure 3: The Qt application at runtime. The left side renders the parking lot polygon and the current pose of the car; the right side groups the controls for both execution paths, the load and store buttons for the Q-table, and the eval checkbox that switches between training and pure greedy evaluation.

4.3 Hyperparameter

All numerical parameters are externalized to `configuration/default.conf`, parsed once at construction time into a `map<string, string>`. Consumers cast values with `stoi` or `stof`, and the constructor calls `setlocale(LC_NUMERIC, "C")` before parsing so that floats use a `.` as decimal separator regardless of system locale. Because the file is read only once, every change requires restarting the application.

The parameters fall into five groups:

- *Car and lot geometry*: `LEN_CAR`, `WIDTH_CAR`, `FRONT_OVERHANG`, `REAR_OVERHANG`, `LEN_ENV`, `WIDTH_ENV`, `LEN_SLOT`, `WIDTH_SLOT`, `FREE_PARK`. These determine the rectangle in Section 2 and the polygon of the parking lot.
- *Reward shaping*: `REWARD_FOR_HIT`, `REWARD_FOR_PARK`, `REWARD_FOR_NOTHING`. The three constants entering the reward function above.
- *Learning hyperparameters*: `Q_LR_MAX`, `Q_LR_MIN`, `Q_LR_HL` for the learning-rate schedule, `ER_MAX`, `ER_MIN`, `ER_HL` for the exploration-rate schedule, and `DISCOUNT_FACTOR` for γ .
- *Discretization*: `X_DIVIDE`, `Y_DIVIDE`, `THETA_DIVIDE` for the state-space grid, `N_SPEED_ACTIONS`, `N_STEERING_ACTIONS`, `STEERING_UNITY` for the action grid.
- *Execution*: `APPROX_MOTION` selects the integrator of Section 2 MSEC, `ANIMATION_SPEED`, and `TIME_RATIO` control the timing of the animated mode.

The Q-table itself is persisted as plain text, with one row per state and `action_count` floats per row separated by spaces. The loader segments rows by column count but does not validate the number of rows against the current discretization. Loading a table trained with different `X_DIVIDE`, `Y_DIVIDE` or `THETA_DIVIDE` therefore silently produces a wrong-shaped table, and consistency is the user's responsibility. The `parameters/` directory is not auto-created either: if it does not exist relative to the working directory, the store operation fails silently with a message on standard error, which on Windows is hidden by default because the `.pro` file does not enable `CONFIG += console`.

5 Experiments and results

The project ships no automated benchmark suite, so evaluation is necessarily qualitative. A configuration is judged by training in headless mode for a fixed number of iterations and then watching, in animated mode with eval enabled, whether the greedy policy parks the car from a sample of random initial poses.

The procedure is reproducible because `set_random_carstate` always resets the heading to $-\frac{\pi}{2}$, which keeps the initial-state distribution narrow and predictable across runs.

5.1 Various hyperparameters and training dimensions tested (maybe removable)

The most consequential design knobs encountered during development are the discretization granularity, the size of the action grid, the reward magnitudes, the learning and exploration schedules, and the choice of integrator. Each of them affects the policy in a qualitatively different way and is worth discussing on its own.

The discretization granularity, set by `X_DIVIDE`, `Y_DIVIDE` and `THETA_DIVIDE`, controls the trade-off between resolution and table size in the most direct way. Coarser grids learn faster because each cell is visited more often, but they cannot distinguish the small alignment changes needed to enter the slot cleanly: two poses that differ by less than a bin width are aliased to the same state and therefore receive the same action regardless of their true geometric relation to the slot. Finer grids increase the table size multiplicatively along three axes, slow convergence, and quickly produce a regime in which most cells are visited rarely or never. The angular resolution `THETA_DIVIDE` deserves particular attention because the parking predicate depends on heading: a $\frac{\pi}{6}$ tolerance around $-\frac{\pi}{2}$ corresponds to roughly one-sixth of the angular range, so an angular grid coarser than that cannot reliably express the difference between an aligned and a misaligned car.

The number and spacing of speed and steering actions plays a similar role on the action side. Too few values prevent the policy from issuing the small corrections needed to centre the body in the slot; too many dilute the learning signal across actions whose effects are nearly indistinguishable at the chosen timestep. Including a zero in both sets is critical: without a “stand still” or “drive straight” action the agent is forced to oscillate around the target instead of settling.

The reward magnitudes set the qualitative shape of the policy. A large $|R_{\text{hit}}|$ relative to R_{park} produces a risk-averse policy that hovers far from walls and rarely reaches the slot at all; a small one yields aggressive behaviour that often clips the pocket corners. The ratio $\frac{R_{\text{hit}}}{R_{\text{park}}}$ matters more than either constant in isolation, and the discount factor γ further interacts with this ratio because it controls how strongly future success is propagated backwards through the Q-table.

The learning and exploration schedules, set by `Q_LR_MAX/MIN/HL` and `ER_MAX/MIN/HL`, determine how quickly the agent transitions from exploration to exploitation. Setting either half-life too short tends to lock the policy into a suboptimal trajectory before the table is informative; setting them too long wastes iterations on random behaviour after the policy has effectively converged. The two schedules are independent, but in practice they are tuned together so that learning-rate decay tracks exploration decay: it makes little sense to stop exploring while still applying large updates.

Finally, the choice of integrator (`APPROX_MOTION`) is a methodological constraint rather than a tuning knob. Training and evaluation must use the same value: a Q-table learned under the linearized model encodes transitions that the exact model does not reproduce, and vice versa. Swapping integrators between training and inference invalidates the policy because the resulting state s' for a given (s, a) is no longer drawn from the distribution the agent was trained on. A reproducible quantitative benchmark would require persisting the headless log in a more structured form (for instance a CSV with one row per logging window), exposing a “success-from- N -random-starts” routine in the user interface, and varying one parameter at a time while holding the others fixed. None of these are implemented in the current codebase.

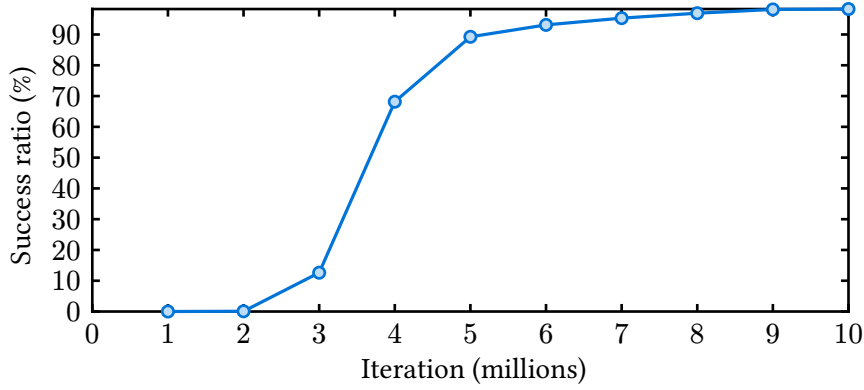


Figure 4: Per-window success ratio from a representative 10-million-iteration training run with the default configuration. Each point reports, over the most recent million iterations, the fraction of terminal events (hit + success) that ended in a successful park. The curve rises as the schedule on ϵ pushes the agent from exploration into exploitation of the learned table.

6 Conclusions

This project shows that a standard tabular Q-learning agent, applied to a properly discretized state space, can learn a parking policy for a kinematic-bicycle car in a constrained two-dimensional lot. The deliberately small scope, that of one Q-table, one car, one slot, made it possible to keep the entire learner, simulator, and visualization in a single Qt application of a few thousand lines, with no external machine-learning dependencies and a fully inspectable internal state.

The exercise also exposes the ergonomic ceiling of tabular methods on this kind of problem. The three discretization axes contribute multiplicatively to the state count, and useful precision near the slot demands fine bins, so the Q-table grows quickly while most of its entries are visited rarely or never. A configuration that parks reliably from one starting region may fail entirely from another simply because the relevant cells have not been updated. The fixed reset orientation of $-\frac{\pi}{2}$ in `set_random_carstate` further narrows the distribution the agent ever observes during training, which simplifies learning at the cost of generality. Despite these limitations the resulting controller is faithful to the textbook formulation of Q-learning and, more importantly, provides a transparent baseline against which any subsequent extension - function-approximation methods, richer reward shaping, more realistic dynamics - can be compared on equal terms.

7 Further Work

The most obvious extension is replacing the tabular Q-table with a function approximator. A small neural network trained with DQN-style targets would eliminate the multiplicative blow-up in the state count, allow continuous (x, y, θ) inputs, and give the agent some capacity to generalize between nearby poses without re-learning each cell separately. The simulator and the reward function would not change, and the comparison against the tabular baseline would be direct. A complementary change on the simulator side would be to introduce simple actuator dynamics: bounded acceleration and a maximum steering rate, instead of the current instantaneous commands, would bring the controlled system closer to a real vehicle and motivate the move to function approximation by enlarging the natural state.

Enriching the reset distribution is a smaller but high-leverage change. The current reset always sets $\theta = -\frac{\pi}{2}$; restoring random orientations, as the commented-out line in `set_random_carstate` already suggests, would force the policy to learn manoeuvres beyond the single-axis approach and expose any latent overfitting to the fixed initial heading. Two infrastructure improvements would in turn make the project quantitatively comparable across configurations: replacing the ad-hoc `log/log12.txt` text file with a structured log of per-window statistics, and adding a “benchmark” mode that runs N greedy episodes from random starts and reports a numerical success rate. Together, these would close the loop on the qualitative discussion of Section 4 and turn it into an objective metric.